

Security Logging Gap Analysis & Remediation Plan

Scope and Purpose

This report answers three questions about the current system (LLDAP · Dex · OpenFGA · Vault · libcloud REST):

1. What security questions must the system be able to answer?
2. Which components can answer each question — and what is already logged vs. what is missing?
3. What concrete changes are needed to close every gap?

Baseline note. The `sysmon2/` design documents (Security Monitoring Architecture, Lightweight Statistics, Nutanix FinOps) are **design-only** — **none are implemented**. No SIEM, log shipper, OpenTelemetry collector, Wazuh, or correlation rules exist today. The current state in §1 is derived from actual code only: the script-level JSONL audit emitters and the libcloud REST `auth_audit` hook.

§1 — Current Logging State (Baseline)

Component	What IS logged today	What is NOT logged (gap)
LLDAP	<code>generated/ldap_audit.log</code> — admin-script actions only (onboard, offboard, group add/remove, group create/delete, password reset); fields: <code>actor</code> , <code>result</code>	Native LDAP bind/search audit — Dex's bind, Vault's bind, user logins; no failed-bind log; no source IP on binds
Dex	Nothing persisted — logs to stdout (glog-style) only	Login success/failure; token issuance (<code>sub</code> , <code>aud</code> , <code>client_id</code> , <code>redirect_uri</code> , IP); refresh-token issuance
OpenFGA server	Nothing structured — container logs to stdout; <code>--log-level / JSON / OTEL</code> not configured in <code>docker-compose.yml</code>	Check decisions (allow/deny, user/relation/object/IP); Write/Delete of tuples; model-id used per call
OpenFGA scripts	<code>generated/openfga_audit.log</code> — tuple-write / tuple-delete only; fields: <code>actor</code> , <code>triple</code> , <code>result</code>	Check decisions; model pushes; store creation; reconciler runs
Vault server	Native audit device NOT enabled — <code>vault-audit-log-query.sh</code> reports "no file audit device configured"	Every request/response (read, write, lease, auth) — critical : Vault blocks all requests it cannot log
Vault scripts	<code>generated/vault_audit.log</code> — every admin script via <code>vault_common.sh</code> ; fields: <code>actor</code> , <code>action</code> , <code>result</code>	Runtime reads by libcloud REST; external access; policy/engine/audit-device mutations
libcloud REST	<code>data/auth_audit.log</code> — only <code>oidc_token_decoded</code> events; fields: <code>principal</code> , <code>issuer</code> , <code>sub</code> , <code>email</code> — no <code>source_ip</code> , no <code>request_id</code>	Per-request access log; OpenFGA denial log; Vault-read log; <code>cloud_request_id</code> ; <code>source_ip</code>
libcloud REST middleware	<code>RequestIDMiddleware</code> sets <code>request.state.request_id + X-Request-ID</code> response header	The ID is not forwarded to OpenFGA, Vault, or cloud calls; no access log consumes it
Host scripts (<code>idp_login.py</code> , <code>set_tenant_credentials.py</code> , <code>superadmin_auth.sh</code>)	Print to stderr only	No structured audit: who logged into Dex, from where, what tenant credentials were written
Cloud providers (AWS CloudTrail, Nutanix Prism Audit)	Native logs exist out-of-band	libcloud REST does not capture cloud-side <code>request_id / task_extId</code> — cloud logs cannot be joined to REST logs
stoplight_mock	N/A (mock, no auth)	—

§2 — Security Questions the System Must Answer

A · Identity & Authentication

ID	Question
A1	Who authenticated to LLDAP (bind), from which IP, when, and did it succeed or fail?
A2	Who logged in to Dex (user, OAuth client, <code>redirect_uri</code> , IP) and was the login successful?
A3	Who obtained a Dex-issued JWT (sub, aud, iss, jti, expiry) and when — including from host scripts?
A4	Who authenticated to Vault via the LDAP auth method (user, policies issued, token TTL, IP)?
A5	Are there brute-force / credential-stuffing patterns against LLDAP, Dex, or Vault?
A6	Did a user authenticate from a new, off-hours, or unexpected IP or host?

B · Authorization & Access Control

ID	Question
B1	For each libcloud REST call: who called, what URL, what provider, what <code>auth_binding</code> , what was the OpenFGA decision, and what was the final HTTP status?
B2	Was an OpenFGA Check denied — for which user/relation/object — and was it followed by a success elsewhere (bypass attempt)?
B3	Were OpenFGA tuples written/deleted outside the reconciler, or by an unexpected actor?
B4	Was the OpenFGA authorization model swapped (model-id mismatch with the pinned id in libcloud REST)?
B5	Did someone self-assign a role (LDAP group add where <code>actor == target_user</code>)?
B6	Was break-glass (<code>superadmin</code>) used, and for which operation?

C · Secrets & Credentials

ID	Question
C1	Who read which Vault secret, when, from which IP, with which token?
C2	Who wrote, rotated, or destroyed a Vault static secret or cloud secrets-engine root credential?
C3	Who requested a dynamic credential (lease id), and was that lease ever renewed or revoked?
C4	Was a cloud credential used outside libcloud REST (cloud provider log has no matching libcloud request)?
C5	Was a credential used after its lease expiry?
C6	Was a Vault policy, LDAP-group binding, secrets engine, or audit device modified?

D · Resource & Cloud Operations

ID	Question
D1	What cloud resources did a user create/modify/destroy, on which tenant/region/cluster, and what was the cloud-side request id?
D2	Was there mass destruction — many destroy operations in a short window?
D3	Was there access to an unexpected cloud region or cluster for that tenant?

E · Lifecycle & Offboarding

ID	Question
E1	Was a user offboarded in LLDAP but still holds OpenFGA tuples, Vault tokens, or leases?
E2	Was a tenant created or deleted, and by whom (must be superadmin-gated)?
E3	Was an LLDAP group created/deleted and a member added/removed, and by whom?

F · Audit Integrity & Defense Evasion

ID	Question
F1	Was the Vault audit device tampered with or disabled?
F2	Was logging itself modified — audit toggle flipped, file path changed, log level lowered?
F3	Are audit logs protected against tampering (append-only, shipped off-host, retained)?

§3 — Question-to-Component Coverage Map

The following table maps each question to which component(s) provide the answer, the current coverage state, and what must be closed.

Q-id	Primary component(s)	Current state	Gap to close
A1	LLDAP (bind audit)	Script actions only; no native bind log	Capture LLDAP stdout; parse <code>bindDN</code> , <code>client_ip</code> , <code>result</code>
A2	Dex (login/token log)	Not persisted	Capture Dex stdout to structured file; parse login + token-issuance events
A3	Dex + libcloud REST + host scripts	REST logs <code>oidc_token_decoded</code> without IP or <code>request_id</code> ; host scripts to stderr only	Add <code>source_ip</code> + <code>request_id</code> to <code>audit_auth_event</code> ; host scripts append to <code>generated/dex_login-audit.log</code>
A4	Vault (native audit device)	Not enabled	<code>vault audit enable file ...</code> in <code>vault_bootstrap.py</code> ; captures <code>auth/ldap/login/*</code> entries
A5	LLDAP + Dex + Vault	No failed-auth correlation source	Solved by A1 + A2 + A4 providing raw events; add failed-bind/login counters
A6	LLDAP + Dex + Vault + libcloud REST	No IP on most events	Solved by A1 + A2 + A4 + B1 (<code>source_ip</code>)
B1	libcloud REST (access log)	Does not exist	New <code>AccessLogMiddleware</code> writing <code>data/rest_access.log</code> with full field set
B2	libcloud REST (denial log)	Does not exist (aspirational <code>generated/openfga_denial.log</code> not wired up)	Emit a denial line from <code>policy.py::_enforce_openfga</code> on every False Check
B3	OpenFGA server (tuple Write/Delete log)	Only admin scripts audit their own writes	Enable OpenFGA structured JSON logging; capture <code>Write/Delete</code> events from server
B4	OpenFGA server + libcloud REST	No model-id pin or check	Log <code>authorization_model_id</code> on every Check; libcloud REST warns on mismatch with pinned id
B5	LLDAP scripts	<code>actor</code> + <code>user</code> logged but no self-assignment detection	Add <code>actor == user</code> check in <code>lldap-group-add-member.sh</code> ; log <code>self_role_assign</code> event at WARN
B6	libcloud REST + OpenFGA	REST sees <code>sub=superadmin</code> but no flag emitted	Tag access log lines with <code>break_glass=true</code> when <code>principal == superadmin</code> ; alert on destructive ops
C1	Vault (native audit device)	Not enabled — libcloud REST runtime reads are invisible	Enable audit device; reads at <code>secret/data/libcloud/*</code> become auditable
C2	Vault (native) + admin scripts	Admin-script actions logged; root-cred rotate logged	Native device also captures REST API writes; <code>set_tenant_credentials.py</code> must append to <code>vault_audit.log</code>
C3	Vault (native)	Not enabled	Audit device logs <code><engine>/creds/<role></code> reads + <code>sys/leases/renew/revoke</code>
C4	Cloud provider logs + libcloud REST	REST does not record <code>cloud_request_id</code>	Capture <code>AWS ResponseMetadata.RequestId</code> / Nutanix task <code>extId</code> into access log; correlate with <code>CloudTrail/Prism</code>

Q-id	Primary component(s)	Current state	Gap to close
C5	Vault + cloud provider	No lease-expiry vs cloud-call correlation	Native audit device + cloud logs (C4) + SIEM rule
C6	Vault (native)	Not enabled	Audit device logs <code>sys/policies/acl/*</code> , <code>sys/mounts/*</code> , <code>sys/audit/*</code> , <code>auth/ldap/groups/*</code>
D1	libcloud REST + cloud provider	No access log; no <code>cloud_request_id</code>	B1 access log + C4 <code>cloud_request_id</code>
D2	libcloud REST	No access log	B1 + SIEM count rule on <code>destroy/delete</code> verbs
D3	libcloud REST + cloud provider	No access log	B1 records <code>provider + auth_binding</code> ; SIEM correlates with cloud region
E1	LLDAP + OpenFGA + Vault	Offboard chain logs each step, but no dangling-state detector	Scheduled offboarding-reconcile job: users disabled in LLDAP with remaining tuples/leases
E2	openfga_my scripts (<code>create_tenant.sh</code>)	Script prints; not audited	Append <code>tenant_create/tenant_delete</code> to <code>generated/tenant_audit.log</code> with superadmin actor + JWT <code>jti</code>
E3	LLDAP scripts	Already audited in <code>lldap_audit.log</code>	No gap
F1	Vault (native)	Not enabled	Audit device logs <code>sys/audit/*</code> writes/deletes
F2	libcloud REST + Vault	<code>auth_audit_enabled</code> toggle not logged on change	Log changes to audit settings at startup and on reload; Vault audit device logs its own config
F3	All	Logs are local files with no rotation or shipping	Add <code>logrotate</code> config + ship to a central collector per <code>sysmon2/</code> target design

§4 — Required Changes

Each change has a priority and effort rating:

- **Priority:** P0 = no logging source exists at all today; P1 = source exists but not useful for correlation; P2 = hardening and SIEM-readiness
- **Effort:** S = hours; M = 1–2 days; L = multi-day

#	Change	Component(s)	Files to touch	Answers	Priority	Effo
1	Add a per-request access log middleware writing one JSONL line per request: <code>request_id</code> , <code>user</code> , <code>source_ip</code> , <code>http_method</code> , <code>path</code> , <code>provider</code> , <code>auth_binding</code> , <code>vault_lease_id</code> , <code>fga_check_result</code> , <code>cloud_request_id</code> , <code>duration_ms</code> , <code>status_code</code> , <code>error</code> , <code>break_glass</code>	libcloud REST	<code>app/common/middleware.py</code> (new <code>AccessLogMiddleware</code>); <code>app/main.py</code> (register); <code>app/config/settings.py</code> (<code>access_log_enabled</code> , <code>access_log_file</code>)	B1, B6, D1, D2, D3, A6	P0	M
2	Propagate <code>request_id</code> to OpenFGA and Vault outbound calls as <code>X-Request-ID</code> , and to AWS/Nutanix driver calls where supported	libcloud REST	<code>app/auth/fga_client.py</code> ; <code>app/connections/vault_client.py</code> ; <code>app/compute/service.py</code> ; <code>app/network/service.py</code> ; <code>app/storage/service.py</code>	B1, B2, C1, C4 (join key)	P0	M
3	Emit an OpenFGA denial log line from <code>policy.py::_enforce_openfga</code> on every False Check: <code>request_id</code> , <code>user</code> , <code>relation</code> , <code>object</code> , <code>scope</code> , <code>reason</code> , <code>source_ip</code> → <code>generated/openfga_denial.log</code>	libcloud REST	<code>app/auth/policy.py</code> ; <code>app/config/settings.py</code> (<code>denial_log_file</code>)	B2, B4	P0	S
4	Add <code>source_ip + request_id</code> to <code>audit_auth_event</code> (<code>oidc_token_decoded</code>)	libcloud REST	<code>app/auth/identity.py</code> (signature + fields); <code>app/auth/oidc_service.py</code> (pass <code>request.client.host + request.state.request_id</code>)	A3, A6	P0	S

#	Change	Component(s)	Files to touch	Answers	Priority	Effo
5	Enable the Vault native audit device (file + syslog) in <code>vault_bootstrap.py</code> with <code>hmac_accessor=false</code> and <code>elide_list_responses=true</code> ; mount a volume for <code>/vault/audit</code>	Vault	<code>../openfga_my/vault_bootstrap.py</code> ; <code>../vault/docker-compose.yml</code> (volume); <code>../vault/ARCHITECTURE.md</code>	A4, C1, C2, C3, C6, F1	P0	M
6	Configure OpenFGA server structured JSON logging + OTEL exporter in <code>docker-compose.yml</code> (<code>--log-level=info</code> + JSON formatter, <code>--otel-collector</code>); capture container stdout to <code>generated/openfga_server.log</code>	OpenFGA	<code>../openfga_my/docker-compose.yml</code> ; <code>setup.sh</code> (log capture)	B1, B3, B4	P1	M
7	Capture Dex login / token-issuance events: persist Dex stdout to <code>generated/dex_server.log</code> and parse login + token endpoints into <code>generated/dex_login_audit.log</code> (<code>user_id, client_ip, redirect_uri, ip, result</code>)	Dex + openfga_my	<code>../dex/docker-compose.yml</code> (logging); <code>../openfga_my/setup.sh</code> (capture); new <code>scripts/dex_login_audit.py</code> (parser)	A2, A3, A5	P1	M
8	Capture LLDAP container stdout to <code>generated/ldap_server.log</code> ; add parser for bind/search/login events into <code>generated/ldap_bind_audit.log</code> (<code>bindDN, client_ip, result</code>) — LLDAP has no OpenLDAP auditlog overlay, so stdout is the only source	LLDAP + openfga_my	<code>../ldap/docker-compose.yml</code> (logging); new <code>scripts/ldap_bind_audit.py</code>	A1, A5, A6	P1	M
9	Capture <code>cloud_request_id</code> from driver responses (AWS <code>ResponseMetadata.RequestId</code> ; Nutanix <code>task_extId</code>) and surface it through the service layer to the access log	libcloud + libcloud REST	<code>libcloud/compute/drivers/nutanix.py</code> (+ AWS EC2 driver); <code>app/compute/service.py</code> , <code>app/network/service.py</code> , <code>app/storage/service.py</code> (return <code>cloud_request_id</code>); <code>app/common/middleware.py</code> (include in log)	C4, D1	P1	M
10	Pin + verify the OpenFGA <code>authorization_model_id</code> in libcloud REST: store the expected id in <code>generated/fga.env</code> ; log + return 503 on mismatch	libcloud REST	<code>app/auth/fga_client.py</code> ; <code>app/config/settings.py</code> (<code>fga_expected_model_id</code>)	B4	P1	S
11	Self-role-assignment detection : in <code>ldap-group-add-member.sh</code> , when <code>actor == user</code> , emit a <code>self_role_assign</code> audit line at WARN; reject unless <code>--allow-self</code> flag is passed	LLDAP scripts	<code>../openfga_my/scripts/ldap-group-add-member.sh</code>	B5	P2	S
12	Host-script audit : <code>idp_login.py</code> , <code>superadmin_auth.sh</code> , <code>set_tenant_credentials.py</code> append a JSONL line to <code>generated/dex_login_audit.log</code> / <code>generated/tenant_audit.log</code> with fields <code>actor, tenant, result, ip</code>	openfga_my scripts	<code>scripts/idp_login.py</code> ; <code>scripts/superadmin_auth.sh</code> ; <code>scripts/set_tenant_credentials.py</code>	A3, E2, C2	P1	S
13	Tenant lifecycle audit : <code>create_tenant.sh</code> (and new <code>delete_tenant.sh</code>) append to <code>generated/tenant_audit.log</code> with <code>superadmin actor + JWT jti</code>	openfga_my scripts	<code>scripts/create_tenant.sh</code> ; new <code>scripts/delete_tenant.sh</code>	E2	P2	S
14	Offboarding dangling-state detector : scheduled job that lists users disabled in LLDAP but still holding OpenFGA tuples / Vault tokens / leases; writes a report to <code>generated/offboard_reconcile.log</code>	openfga_my	new <code>scripts/offboarding-reconcile.sh</code> + cron/systemd timer	E1	P2	M

#	Change	Component(s)	Files to touch	Answers	Priority	Effo
15	Settings-change audit: log any change to <code>auth_audit_enabled</code> , <code>auth_audit_file</code> , <code>access_log_enabled</code> , <code>denial_log_file</code> , <code>fga_expected_model_id</code> at startup and on reload	libcloud REST	<code>app/config/settings.py</code> (startup audit line); <code>app/common/middleware.py</code>	F2	P2	S
16	Log rotation + retention: <code>logrotate</code> config for all generated <code>*_audit.log</code> , <code>data/auth_audit.log</code> , <code>data/rest_access.log</code> , <code>generated/openfga_denial.log</code> , <code>/vault/audit/audit.log</code> per <code>sysmon2/</code> retention table (Vault: 3yr; REST: 3yr; FGA: 1yr; LLDAP: 3yr)	All	<code>new ops/logrotate.d/libcloud</code> ; <code>setup.sh</code> install step	F3	P2	S
17	Append-only / tamper evidence: write audit files with <code>chattr +a</code> (Linux) where the host supports it, or ship via append-only syslog to a remote collector	All	<code>setup.sh</code> (post-create <code>chattr +a</code>); <code>ops/logrotate.d/libcloud</code> (<code>copytruncate</code>)	F3	P2	S
18	break-glass tagging: set <code>break_glass=true</code> in access log / denial log / auth audit when <code>principal == superadmin</code> ; add SIEM rule for destructive ops under break-glass	libcloud REST	<code>app/common/middleware.py</code> ; <code>app/auth/policy.py</code> ; <code>app/auth/identity.py</code>	B6	P2	S
19	Reconciler audit: <code>openfga-tuple-reconcile.py</code> writes a summary line (added/removed/unchanged counts + actor) to <code>generated/openfga_audit.log</code> so reconciler-driven writes are distinguishable from direct writes	openfga_my	<code>scripts/openfga-tuple-reconcile.py</code>	B3	P2	S
20	Denial-query correlation: extend <code>openfga-denial-log-query.sh</code> to correlate denial entries with the matching REST access-log line by <code>request_id</code> (currently only queries OpenFGA-side denials)	openfga_my	<code>scripts/openfga-denial-log-query.sh</code>	B2	P2	S

§5 — Priority Summary

P0 — Blind spots with no logging source today

All five changes below must land before any security monitoring is meaningful.

#	Change	Covers
1	REST access log middleware	B1, B6, D1, D2, D3, A6
2	<code>request_id</code> propagation to OpenFGA + Vault + cloud	B1, B2, C1, C4
3	OpenFGA denial log from <code>policy.py</code>	B2, B4
4	<code>source_ip</code> + <code>request_id</code> in auth audit event	A3, A6
5	Vault native audit device (file + syslog)	A4, C1, C2, C3, C6, F1

P1 — Source exists but not usable for correlation

#	Change	Covers
6	OpenFGA server structured JSON + OTEL	B1, B3, B4
7	Dex login/token capture + parser	A2, A3, A5

#	Change	Covers
8	LLDAP stdout capture + bind audit parser	A1, A5, A6
9	cloud_request_id from driver responses	C4, D1
10	FGA model-id pin + mismatch detection	B4
12	Host-script audit lines	A3, E2, C2

P2 — Hardening and SIEM-readiness

#	Change	Covers
11	Self-role-assignment detection	B5
13	Tenant lifecycle audit	E2
14	Offboarding dangling-state detector	E1
15	Settings-change audit	F2
16	Log rotation + retention	F3
17	Append-only / tamper evidence	F3
18	break_glass tagging	B6
19	Reconciler audit summary	B3
20	Denial-query cross-log correlation	B2

§6 — Suggested Implementation Sequence

The following sequence minimises effort while maximising coverage at each step.

1. **Land #5 first (Vault audit device)** — single infrastructure change; turns C1/C2/C3/C6/F1 from completely blind to fully covered with zero application code changes.
2. **Land #1 + #2 + #3 + #4 together** — forms the libcloud REST correlation fabric (access log + request_id propagation + denial log + source_ip). After this sprint, questions B1/B2/B6/D1/D2/D3/A3/A6 are all answerable.
3. **Land #6 + #7 + #8 + #9 + #10 + #12** — server-side structured logs for OpenFGA/Dex/LLDAP, cloud_request_id capture, model-id pin, and host-script audit. After this sprint, every question in §2 has at least one explicit logging source.
4. **Land the P2 hardening batch (#11, #13–#20)** — integrity, retention, detection convenience.

§7 — Per-Component Instrumentation Checklist

Component	Changes owed (§4 reference)
libcloud REST	#1 access log middleware · #2 request_id propagation · #3 denial log · #4 source_ip+request_id in auth audit · #9 cloud_request_id capture · #10 model-id pin · #15 settings-change audit · #18 break_glass tagging
Vault	#5 native audit device + volume mount
OpenFGA server	#6 structured JSON logging + OTEL export
Dex	#7 login/token capture (stdout → file)
LLDAP	#8 stdout capture + bind audit parser
libcloud (drivers)	#9 cloud_request_id return (Nutanix extId + AWS RequestId)
openfga_my scripts	#7 parser · #8 parser · #11 self-role detection · #12 host-script audit · #13 tenant audit · #14 offboarding reconciler · #19 reconciler audit · #20 denial-query correlation

Component	Changes owed (\$4 reference)
ops / <u>setup.sh</u>	#16 logrotate config · #17 append-only (chat tr +a) + syslog ship